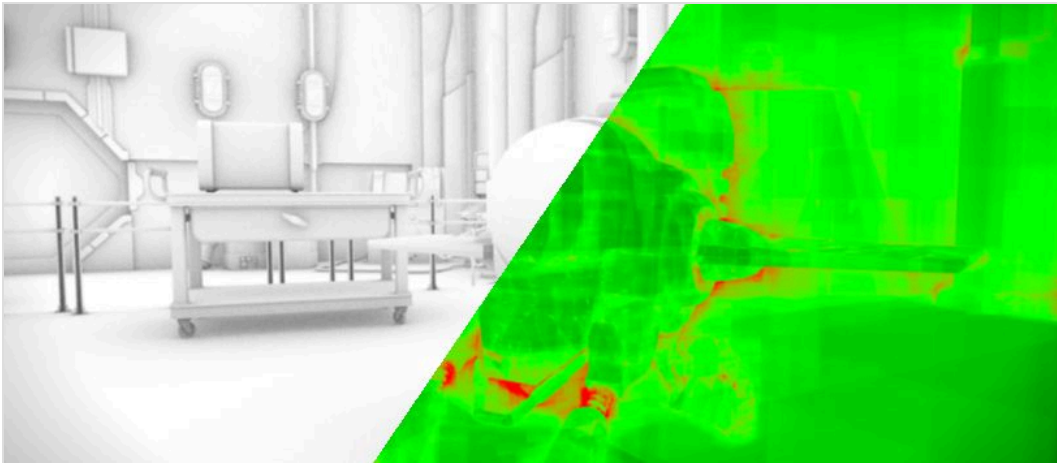




How to build a BVH – Part 1: Basics

② JBikker ① APRIL 13, 2022 ② 22 COMMENTS 📁 ARTICLE, CODING, SOURCE CODE, TUTORIAL

In this series we explore the wonderful world of the *bounding volume hierarchy*: the mysterious data structure that enables real-time ray tracing on recent GPUs, in complex, animated scenes. Despite the broad adoption of ray tracing in games, in-depth understanding of the underlying technology seems to be reserved to a select few. In this article we show how a proper BVH is implemented without support from DXR / RTX / OptiX / Vulkan, in easy-to-read ‘sane C++’. Whether this is actually useful for you, or just interesting as background information, depends on your needs, obviously.

This series consist of ten articles. For each article, full source code is [available on Github](#), in easy-to-read ‘Sane C++’. Contents:

1. **Basics: data structure, simple construction and ray traversal (this article).**
2. [Faster rays: the Surface Area Heuristic and ordered traversal.](#)
3. [Faster construction: binned BVH building.](#)
4. [BVHs for animated models: refitting and rebuilding.](#)
5. [BVHs for animated scenes: the top-level BVH.](#)
6. [TLAS & BLAS part 2 – all together now.](#)
7. [Consolidating, part 1: ray tracing textured and shaded meshes.](#)
8. [Consolidating, part 2: Whitted-style ray tracing.](#)
9. [OpenCL: GPU ray tracing, part 1.](#)
10. [OpenCL: GPU ray tracing, part 2 \(series finale\).](#)

In this first article we discuss the bare basics: what the BVH is used for, and how a basic BVH is constructed and traversed with a ray. The end product is



Recent Posts

Rendering for DELTA –
3D on MSX2

BVH Quality: Beyond
SBVH

tinybvH – Manual –
Advanced

tinybvH – Manual –
Basic Use

Ray Tracing with Voxels
in C++ Series – Part 6

Recent Comments

Sebastian on tinybvH –
Manual – Advanced

jbikker on tinybvH –
Manual – Advanced

Sebastian on tinybvH –
Manual – Advanced

jbikker on tinybvH –
Manual – Advanced

Sebastian on tinybvH –
Manual – Advanced

Archives

March 2026

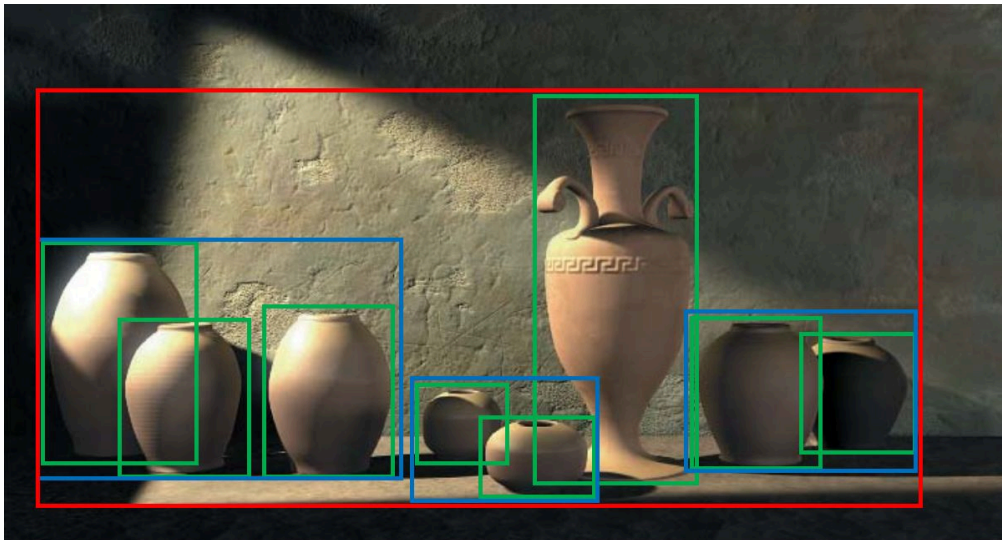
May 2025

January 2025



Background

The BVH is an *acceleration structure*. It serves to reduce a pretty fundamental problem in ray tracing: the sheer cost of finding the nearest intersection of a ray and a scene consisting of (potentially) millions of polygons. The basic principle is easy to understand: if we have 64 polygons, we could split them into two groups of 32 polygons, each group enclosed by a *bounding volume* such as a box or a sphere. When intersecting the scene, we can skip 32 polygons if the ray does not intersect the enclosing volume, nearly halving the processing time. Next, we do the same to each of the two groups: split in two halves, encapsulate, recurse. The resulting bounding volume *hierarchy* brings down processing time to manageable levels: with each level, we halve the number of polygons. Starting with 64 polygons, we traverse a tree in six steps to arrive at a single polygon.



BVH: a hierarchy of bounding volumes. We can skip any volume that the ray doesn't intersect. Image based on *A 3-Dimensional Representation for Fast Rendering of Complex Scenes*, by Rubin and Whitted.

That's the global idea, but let's make things practical. I will be using a [minimalistic template](#) from one of my courses for these experiments, but the code is designed to be generic. Feel free to use something else.

We start with the definition of a triangle:

```
1 | struct Tri { float3 vertex0, vertex1, vertex2; float3 centroid; }
```

Your triangle may need more properties for rendering, but for building a BVH and intersecting the triangle with a ray, this is all we need. Speaking of rays:

```
1 | struct Ray { float3 O, D; float t = 1e30f; };
```

April 2024

January 2024

September 2023

August 2023

June 2022

May 2022

April 2022

August 2021

February 2021

May 2020

April 2020

March 2020

December 2019

July 2019

May 2019

March 2019

Categories

Article

Coding

Demo

MSX

Source Code

Tutorial

Uncategorized

Meta

Log in

Entries feed

Comments feed

to 'infinity' (well, almost) here.

Let's construct some random triangles to play with:

```
1 // triangle count
2 #define N 64
3 Tri tri[N];
4 // application init; gets called once at startup
5 void BasicBVHApp::Init()
6 {
7     for (int i = 0; i < N; i++)
8     {
9         float3 r0( RandomFloat(), RandomFloat(), RandomFloat() )
10        float3 r1( RandomFloat(), RandomFloat(), RandomFloat() )
11        float3 r2( RandomFloat(), RandomFloat(), RandomFloat() )
12        tri[i].vertex0 = r0 * 9 - float3( 5 );
13        tri[i].vertex1 = tri[i].vertex0 + r1;
14        tri[i].vertex2 = tri[i].vertex0 + r2;
15    }
16 }
```

Here, `RandomFloat()` yields a uniform random number in the range 0..1. I used [Marsaglia's Xor32 RNG](#), but anything will do. Given three vectors with random components, a triangle is then produced within a 10x10x10 cube centered around the origin. Not pretty, but good enough for now. Feel free to replace this with a proper model, e.g. imported using [assimp](#) or [TinyOBJ](#).

Next, we need a way to intersect a triangle with a ray:

```
1 void IntersectTri( Ray& ray, const Tri& tri )
2 {
3     const float3 edge1 = tri.vertex1 - tri.vertex0;
4     const float3 edge2 = tri.vertex2 - tri.vertex0;
5     const float3 h = cross( ray.D, edge2 );
6     const float a = dot( edge1, h );
7     if (a > -0.0001f && a < 0.0001f) return; // ray parallel to
8     const float f = 1 / a;
9     const float3 s = ray.O - tri.vertex0;
10    const float u = f * dot( s, h );
11    if (u < 0 || u > 1) return;
12    const float3 q = cross( s, edge1 );
13    const float v = f * dot( ray.D, q );
14    if (v < 0 || u + v > 1) return;
15    const float t = f * dot( edge2, q );
16    if (t > 0.0001f) ray.t = min( ray.t, t );
17 }
```

This is the famous [Möller-Trumbore intersection algorithm](#), which is still pretty close to 'as fast as possible'. There exist [slightly faster approaches](#) (see Chapter 5 of Sven's thesis), but these need more (or rather, different) data than just the three vertices.

The `IntersectTri` function determines if there is an intersection between the ray and the triangle. If this is the case, we check if the found intersection distance is closer than what the ray encountered before. If so, the `t` value for the ray is updated. For a given ray, we can now simply check

Archives

March 2026

May 2025

January 2025

May 2024

April 2024

January 2024

September 2023

August 2023

June 2022

May 2022

April 2022

August 2021

February 2021

May 2020

April 2020

March 2020

December 2019

July 2019

May 2019

March 2019

Pages

Articles

DELTA (MSX2)

Lighthouse 2

Personal

Projects

Publications

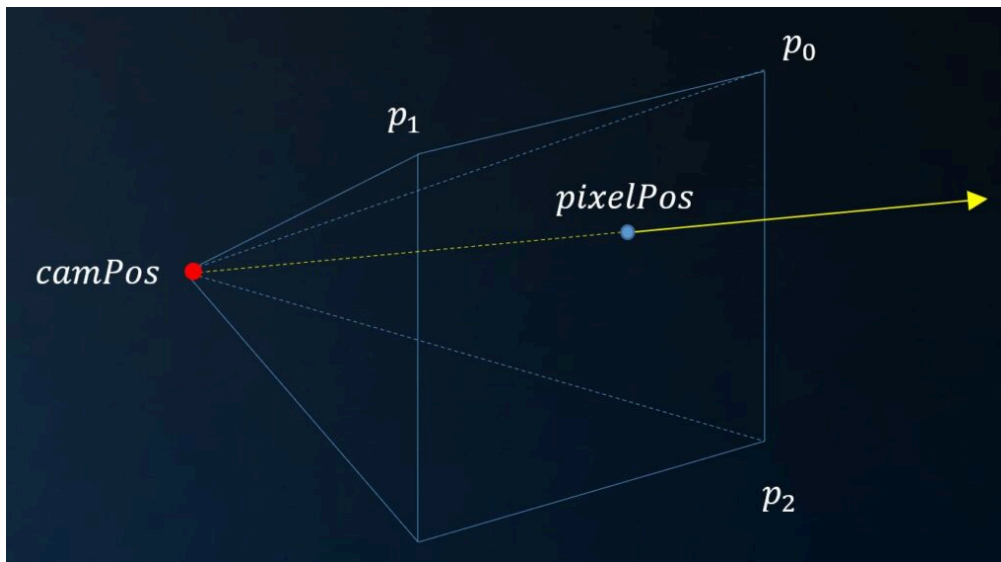
So, let's generate some rays. For e.g. a 640 x 640 image, we do this by firing rays from a camera position to a virtual screen plane:

```

1 float3 camPos( 0, 0, -18 );
2 float3 p0( -1, 1, -15 ), p1( 1, 1, -15 ), p2( -1, -1, -15 );
3 Ray ray;
4 for (int y = 0; y < 640; y++) for (int x = 0; x < 640; x++)
5 {
6     float3 pixelPos = p0 + (p1 - p0) * (x / 640.0f) + (p2 - p0) *
7     ray.o = camPos;
8     ray.D = normalize( pixelPos - ray.o );
9     ray.t = 1e30f;

```

Here, the camera is placed at position (0,0,-18), so: well outside the cube that contains the triangles. Virtual screen corners are located at (-1, 1, -15), (1,1,-15) and (-1,-1,-15), so 3 units in front of the camera. For each of the pixels we can now calculate a position on this square. The ray is then initialized with the camera position as origin, and a normalized vector from the ray origin to the pixel position as a direction.



Generating primary rays: pixelPos is obtained by interpolating over the rectangle defined by p0, p1 and p2.

With a properly initialized primary ray we can now get our first ray traced view of the test scene, by 'brute force' checking the ray against every triangle. The result is a *t* value in the ray for each pixel. This value is 1e30f if we hit nothing, or the actual distance to a triangle otherwise.

```

1 for( int i = 0; i < N; i++ ) IntersectTri( ray, tri[i] );

```



Result of ray tracing 64 triangles.

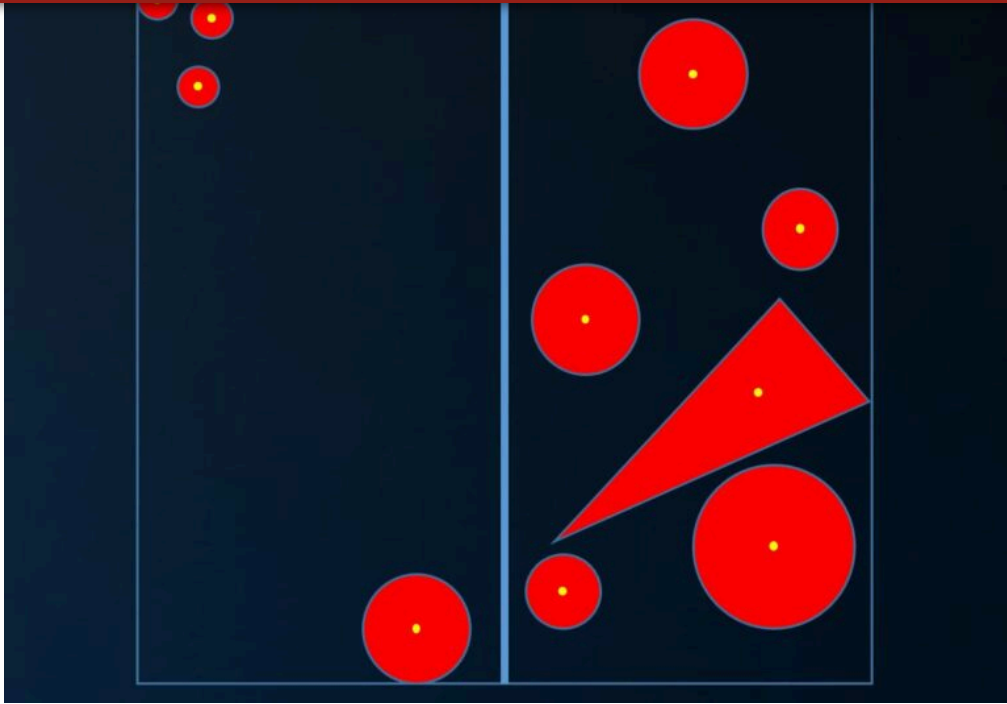
The result is underwhelming, but hey, it shows that the basic setup works. I'll leave the explanation of proper Whitted-style ray tracing to [Peter Shirley](#)'s amazing book and focus on the efficiency of those rays instead.

Concepts and Ingredients

On my machine, it takes about 140 milliseconds to construct the 640 x 640 rays and intersect them with the 64 triangles. That's not bad by the way: we trace almost 3 million rays per second, and that's not even using multithreading or the GPU. Thing is, however: computation scales linearly in the number of triangles... Tracing 256 triangles takes 540 milliseconds; 1024 triangles bring this to 2250 milliseconds.

Time to introduce the BVH.

When Rubin & Whitted first described the bounding volume hierarchy, they assumed the bounding volumes would be placed manually. This is not something artists will enjoy, so let's see how a BVH can be constructed automatically.

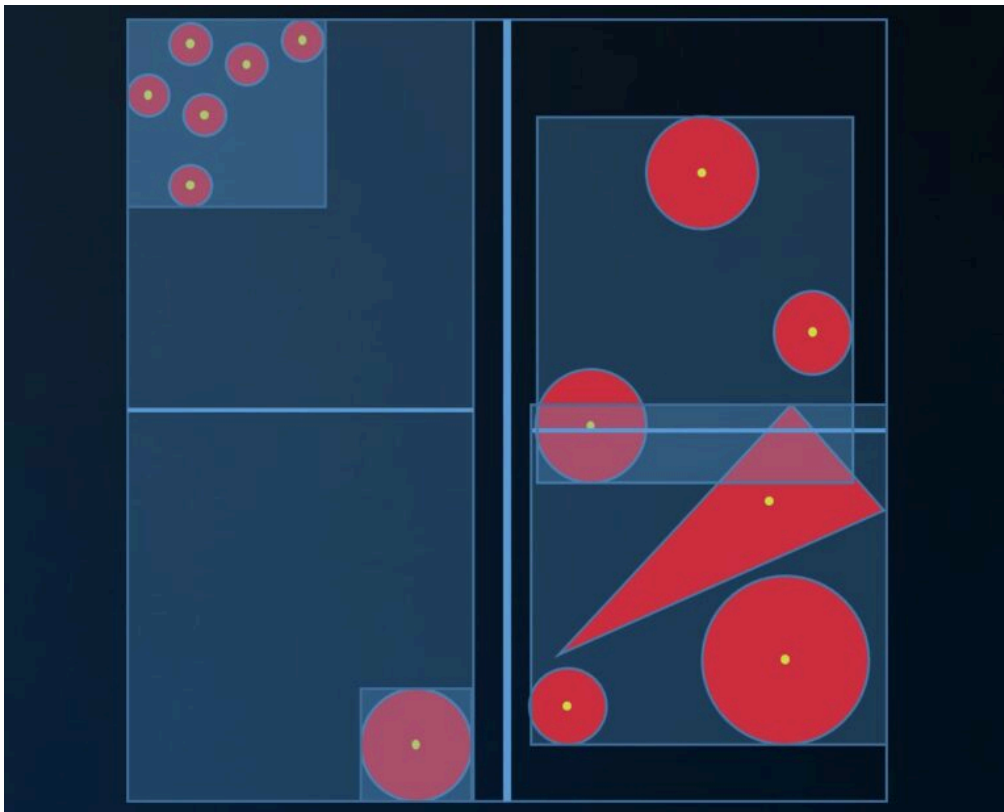


We start with the scene and its bounding box. This is going to be the root node for the hierarchy. We then recursively subdivide this node. A *split plane* partitions the primitives in two sets. Note that we do not split *space*: this might lead to primitives being split and/or assigned to both sides. In a BVH this never happens. The partitioning is what we need the primitive *centroids* for: each triangle (or disc, in the figure above) is either to the left of the split plane, or *not* to the left (which includes the situation where the primitive is *on* the split plane – we need to be precise!).

After the split we have two (now independent) groups of objects, with their updated group bounds:



Note that the bounding volumes of the two groups may overlap. This actually happens when we split the right group vertically:



Splitting the left group also leads to an interesting situation: the group of discs at the top and the single disc at the bottom have bounds that are pretty far apart. A ray that does intersect the left group may thus miss both of its child nodes, avoiding intersection tests with seven discs.

For a BVH node we need the following data:

in 3D space. Note: a sphere would be even cheaper to store, but it is not as convenient to work with in practice.

2. References to the two child nodes, if the node is not a leaf node.
3. References to primitives, if the node is a leaf node.

A naïve BVH node layout would look something like:

```
1 struct BVHNode
2 {
3     aabb nodeBounds;
4     BVHNode* left, *right;
5     bool isLeaf;
6     vector<Tri*> primitives;
7 };
```

However, this struct has many problems. It is rather large, but also somewhat unpredictable in size due to the vector and the pointers, which take up 4 bytes on a 32-bit OS, but 8 on a 64-bit OS.

The first improvement is switching from pointers to *indices*. For triangles we use their indices in the triangle array; for child node pointers we can do something similar if we pre-allocate all nodes that we ever need in a BVHNode pool. We can do this, because the size of the BVH for N triangles has an upper limit: we can never have more than $2N-1$ nodes, since N primitives in N leaves have no more than $N/2$ parents, $N/4$ grandparents and so on.

With primitive pointers and BVHNode pointers replaced by indices the BVHNode struct reduces to:

```
1 struct
2 {
3     float3 aabbMin, aabbMax; // 24 bytes
4     uint leftChild, rightChild; // 8 bytes
5     bool isLeaf; // 4 bytes
6     uint firstPrim, primCount; // 8 bytes; total: 44 bytes
7 };
```

Note how we cleverly store a list of primitives using just two numbers: we simply store the index of the first primitive, and the number of primitives. This way, any number of primitives can be specified in just eight bytes, as long as they are stored consecutively in the primitive array. It turns out that we can enforce this elegantly.

We can make several improvements; most of these I will mention later but one is useful now: boolean `isLeaf` is redundant once we realize that for a leaf the `primCount` cannot be zero, while for all other nodes `leftChild` and `rightChild` cannot be zero. This yields (for now) the final structure with a size of 40 bytes.

Construction


```

1  BVHNode bvhNode[N * 2 - 1];
2  uint rootNodeIdx = 0, nodesUsed = 1;
3
4  void BuildBVH()
5  {
6      for (int i = 0; i < N; i++) tri[i].centroid =
7          (tri[i].vertex0 + tri[i].vertex1 + tri[i].vertex2) * 0.3
8          // assign all triangles to root node
9      BVHNode& root = bvhNode[rootNodeIdx];
10     root.leftChild = root.rightChild = 0;
11     root.firstPrim = 0, root.primCount = N;
12     UpdateNodeBounds( rootNodeIdx );
13     // subdivide recursively
14     Subdivide( rootNodeIdx );
15 }

```

The first lines of this function initialize the centroids for the triangles, which are needed for the subdivision process. After that, we claim one node from the bvhNode array. The root node is initialized: the child references are zeroed, and all primitives are added to it.

Updating the AABB for the root node is straightforward:

```

1  void UpdateNodeBounds( uint nodeId )
2  {
3      BVHNode& node = bvhNode[nodeId];
4      node.aabbMin = float3( 1e30f );
5      node.aabbMax = float3( -1e30f );
6      for (uint first = node.firstPrim, i = 0; i < node.primCount;
7          {
8          Tri& leafTri = tri[first + i];
9          node.aabbMin = fminf( node.aabbMin, leafTri.vertex0 );
10         node.aabbMin = fminf( node.aabbMin, leafTri.vertex1 );
11         node.aabbMin = fminf( node.aabbMin, leafTri.vertex2 );
12         node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex0 );
13         node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex1 );
14         node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex2 );
15     }
16 }

```

Note: The fminf and fmaxf functions used here return the minimum and maximum x, y and z for pairs of float3's. Each vertex of each triangle in the BVHNode is visited to find the lowest and highest x, y and z: the resulting set of six values is the AABB we are looking for.

Finally, we recursively subdivide the root node. A few steps are required:

1. Determine the axis and position of the split plane
2. Split the group of primitives in two halves using the split plane
3. Create child nodes for each half
4. Recurse into each of the child nodes.

1. Split plane axis and position: for now, we will split the AABB along its longest axis. Later we will discuss better ways, but for now this is fine. Code:

```

1  float3 extent = node.aabbMax - node.aabbMin;
2  int axis = 0;
3  if (extent.y > extent.x) axis = 1;

```

2. Split the group in two halves: this sounds like a complicated thing to do, but in BVH construction this is surprisingly simple. Because we are not splitting any triangles, the combined size of the list of triangles in both halves is exactly the size of the unsplit list. That means the split can be done *in place*. For this, we walk the list of primitives, and swap each primitive that is not on the left of the plane with a primitive at the end of the list. This is functionally equivalent to a QuickSort partition.

```
1  int i = node.firstPrim;
2  int j = i + node.primCount - 1;
3  while (i <= j)
4  {
5      if (tri[i].centroid[axis] < splitPos)
6          i++;
7      else
8          swap( tri[i], tri[j--] );
9  }
```

Note that the triangles do not need to be sorted; it is sufficient that the two groups of triangles on both sides of the split plane are stored consecutively.

3. Creating child nodes for each half: Using the outcome of the partition loop we construct two child nodes. The first child node contains the primitives at the start of the array; the second child node contains the primitives starting at *i*. The number of primitives on the left is thus *i - node.firstPrim*; the right child has the remaining primitives. In code:

```
1  int leftCount = i - node.firstPrim;
2  if (leftCount == 0 || leftCount == node.primCount) return;
3  // create child nodes
4  int leftChildIdx = nodesUsed++;
5  int rightChildIdx = nodesUsed++;
6  node.leftNode = leftChildIdx;
7  bvhNode[leftChildIdx].firstPrim = node.firstPrim;
8  bvhNode[leftChildIdx].primCount = leftCount;
9  bvhNode[rightChildIdx].firstPrim = i;
10 bvhNode[rightChildIdx].primCount = node.primCount - leftCount;
11 node.primCount = 0;
```

Some details to point out here:

- Theoretically, the split in the middle can yield an empty box on the left or the right side. Not easy to come up with such a situation though! Left as an exercise for the reader. 😊
- We should not forget to set the primitive count of the node we just split to zero. Since we do not store the `isLeaf` boolean anymore, we rely on the `primCount` to identify leaves and interior nodes.
- Since we increment the 'next free bvh node counter' `nodesUsed` twice on consecutive lines, it is obvious that `rightChildIdx` is always equal to `leftChildIdx + 1`. This of course should have

We're almost done with the subdivide functionality. One we have generated the left and right child nodes, we update their bounds (using a call to `UpdateNodeBounds`, which we created earlier), and we recurse into the child nodes (using a call to the `Subdivide` function we're currently writing). Leaves us with one question: when does the recursion end? One would say: once there is one primitive left. This is sadly not true. The point is, two triangles quite often form a quad in real-world scenes. If this quad is axis-aligned, there is no way we can split it (with an axis-aligned plane) into two non-empty halves. For this reason, we typically terminate when we have 2 or less primitives in a leaf. Even this is not completely safe – but for a solution we need a seemingly unrelated technique, which will be discussed in the next article.

One More Thing

There is one nagging oversight in the partition code that we need to fix. I am talking about this line:

```
1 | swap( tri[i], tri[j--] );
```

This line has two issues. First, we are changing the original list of triangles. This may be fine if it is owned by the BVH builder, but not so great if it is shared with e.g. the animation system of an engine. Secondly, we are swapping *whole triangles* by value. Their size is somewhat limited in this case, but what if a triangle has uv-coordinates, material ids, a name and so on?

We can resolve these issues by using an intermediate array with triangle indices. It contains simple unsigned integers and gets initialized with the numbers $0..N-1$. From then on, we swap entries in this array, rather than the triangles themselves. A `BVHNode` leaf now references a slice of indices, rather than a set of consecutive triangles. It is an extra step of indirection, but it proves to be quite practical.

We make a few small changes to facilitate this new data structure:

```
1 | Tri tri[N];  
2 | uint triIdx[N];
```

At the start of `BuildBVH` we initialize this array:

```
1 | void BuildBVH()  
2 | {  
3 |     // populate triangle index array  
4 |     for (int i = 0; i < N; i++) triIdx[i] = i;  
5 |     ....
```

```
1 struct BVHNode
2 {
3     float3 aabbMin, aabbMax;
4     uint leftNode, firstTriIdx, triCount;
5     bool isLeaf() { return triCount > 0; }
6 };
```

UpdateBounds now uses the new indirection:

```
1 void UpdateNodeBounds( uint nodeIdx )
2 {
3     BVHNode& node = bvhNode[nodeIdx];
4     node.aabbMin = float3( 1e30f );
5     node.aabbMax = float3( -1e30f );
6     for (uint first = node.firstTriIdx, i = 0; i < node.triCount
7         {
8         uint leafTriIdx = triIdx[first + i];
9         Tri& leafTri = tri[leafTriIdx];
10        node.aabbMin = fminf( node.aabbMin, leafTri.vertex0 ),
11        node.aabbMin = fminf( node.aabbMin, leafTri.vertex1 ),
12        node.aabbMin = fminf( node.aabbMin, leafTri.vertex2 ),
13        node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex0 ),
14        node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex1 ),
15        node.aabbMax = fmaxf( node.aabbMax, leafTri.vertex2 );
16    }
17 }
```

The full Subdivide function, of which we have seen bits and pieces so far, in its full glory:

```
1 void Subdivide( uint nodeIdx )
2 {
3     // terminate recursion
4     BVHNode& node = bvhNode[nodeIdx];
5     if (node.triCount <= 2) return;
6     // determine split axis and position
7     float3 extent = node.aabbMax - node.aabbMin;
8     int axis = 0;
9     if (extent.y > extent.x) axis = 1;
10    if (extent.z > extent[axis]) axis = 2;
11    float splitPos = node.aabbMin[axis] + extent[axis] * 0.5f;
12    // in-place partition
13    int i = node.firstTriIdx;
14    int j = i + node.triCount - 1;
15    while (i <= j)
16    {
17        if (tri[triIdx[i]].centroid[axis] < splitPos)
18            i++;
19        else
20            swap( triIdx[i], triIdx[j--] );
21    }
22    // abort split if one of the sides is empty
23    int leftCount = i - node.firstTriIdx;
24    if (leftCount == 0 || leftCount == node.triCount) return;
25    // create child nodes
26    int leftChildIdx = nodesUsed++;
27    int rightChildIdx = nodesUsed++;
28    bvhNode[leftChildIdx].firstTriIdx = node.firstTriIdx;
29    bvhNode[leftChildIdx].triCount = leftCount;
30    bvhNode[rightChildIdx].firstTriIdx = i;
31    bvhNode[rightChildIdx].triCount = node.triCount - leftCount;
32    node.leftNode = leftChildIdx;
33    node.triCount = 0;
34    UpdateNodeBounds( leftChildIdx );
```

```

38     Subdivide( rightChildIdx );
39 }

```

And there you have it: one valid BVH constructed from a polygon soup, in a (perhaps) surprisingly short snippet of code.

Traverse

Time to put the BVH to work. Traversing it is of course a recursive process, starting at the root node:

1. Terminate if the ray misses the AABB of this node.
2. If the node is a leaf: intersect the ray with the triangles in the leaf.
3. Otherwise: recurse into the left and right child.

That doesn't sound too complex, and indeed: it is implemented in just a few lines of code.

```

1 void IntersectBVH( Ray& ray, const uint nodeId )
2 {
3     BVHNode& node = bvhNode[nodeId];
4     if (!IntersectAABB( ray, node.aabbMin, node.aabbMax )) return
5     if (node.isLeaf())
6     {
7         for (uint i = 0; i < node.triCount; i++)
8             IntersectTri( ray, tri[triIdx[node.firstTriIdx + i]]
9         }
10    else
11    {
12        IntersectBVH( ray, node.leftNode );
13        IntersectBVH( ray, node.leftNode + 1 );
14    }
15 }

```

The IntersectAABB function is one function we did not implement yet. To detect if a ray intersects an AABB we use the *slab test*. Note that we don't care *where* we hit the box; the only info we need is a yes/no answer.

Without further explanation:

```

1 bool IntersectAABB( const Ray& ray, const float3 bmin, const flo
2 {
3     float tx1 = (bmin.x - ray.O.x) / ray.D.x, tx2 = (bmax.x - ra
4     float tmin = min( tx1, tx2 ), tmax = max( tx1, tx2 );
5     float ty1 = (bmin.y - ray.O.y) / ray.D.y, ty2 = (bmax.y - ra
6     tmin = max( tmin, min( ty1, ty2 ) ), tmax = min( tmax, max(
7     float tz1 = (bmin.z - ray.O.z) / ray.D.z, tz2 = (bmax.z - ra
8     tmin = max( tmin, min( tz1, tz2 ) ), tmax = min( tmax, max(
9     return tmax >= tmin && tmin < ray.t && tmax > 0;
10 }

```

The IntersectBVH function now does the work we previously did with the loop over all the triangles. That means that this loop now simply gets replaced by a single function call:

```

1 #if 0
2     for( int i = 0; i < N; i++ ) IntersectTri( ray, tri[i] );
3 #else
4     IntersectBVH( ray, rootNodeIdx );

```

Well, on my machine I got 140 milliseconds for brute-force intersecting 64 triangles. With the BVH, this drops to (drumroll...) 32 milliseconds; a 4.4x speedup. Things get more interesting at 1024 triangles. At this scene size, frame time without BVH is 2250 milliseconds; with BVH this drops to 112 milliseconds. That is 20x faster. For larger scenes, the brute force approach becomes infeasible. And we're only getting started: better BVHs will yield faster rays.

Dangling Pointers

In this article I postponed a lot of things. For instance: how can we make the BVHNode structure even smaller, and why should we do this? And what about the 'ideal' split plane axis and position? How do we safely terminate recursion in Subdivide? And what about multithreading construction?

Most of these topics are for another day, but let's end this article with a better BVHNode structure. We currently have:

```
1 struct BVHNode
2 {
3     float3 aabbMin, aabbMax;
4     uint leftNode, firstTriIdx, triCount;
5 };
```

This takes 36 bytes. That means that, on a modern GPU, one node fits in a 64-byte cache line, but quite frequently a node will in fact be partially in one cache line and partially in the next. This is a pretty severe performance issue. If we can reduce the BVHNode to 32 bytes, and if we align the array of nodes that we pre-allocate to a 64-byte boundary in memory (e.g. using [aligned_malloc](#)), then *every* node always uses exactly half of a cache line. Even better: if we are careful, the two children of a node will be in the *same* cache line (*for the impatient: skip one node after the root to achieve this*).

We can reach the 32-byte node size when we realize that a node that has triangles *must* be a leaf. It thus has no child nodes. And the other way round: if there are child nodes, there are no triangles. We thus need to store a child node index **or** the index of the first triangle. That means these can be stored in one and the same variable:

```
1 struct BVHNode
2 {
3     float3 aabbMin, aabbMax;
4     uint leftFirst, triCount;
5 };
```

How we interpret this mysterious 'leftFirst' variable depends on triCount. If it is 0, leftFirst contains the index of the left child node. Otherwise, it contains the index of the first triangle index.

[Continue reading in article 2](#), in which we explore how to trace rays more quickly.

Thanks

This article now has a small but important fix in the ray traversal code, as suggested by Daniel Moore. Thanks!

Closing Remarks

The complete code for this project (for Windows / Visual Studio) [can be found on GitHub](#).

Enjoyed the article? Spotted an error?

Ping me on Twitter ([@j_bikker](#)), comment on this article, or send an email to bikker.j@gmail.com.

You can also find me on Github: <https://github.com/jbikker>.

Or read some more articles: <https://jacco.ompf2.com>. Some favorites:

- The [next article in this series](#), on faster ray traversal.
- [Wavefront path tracing](#)
- Probability Theory for Physically Based Rendering – [part 1](#)
- Probability Theory for Physically Based Rendering – [part 2](#)



Author: jbikker

← How to build a BVH – part 2: Faster Rays

3D Game Programming in C++, Part 3: Variables →

22 thoughts on “How to build a BVH – Part 1: Basics”



Division says:

September 18, 2022 at 9:25 am

Extremely useful articles! Thanks a lot for the material!

Somehow I got an empty box on one side right away on my test mesh so made a workaround by trying other axis if this happens.

Reply



Thanks a ton for these articles 🙏

I wanted to share a nice finding, using axis of greatest variance works really well with meshes that contain long thin triangles, while simply using axis of largest bounding box side causes a leaf node to contain thousands of triangles which makes traversal very slow.

Reply



jbikker says:

December 19, 2022 at 6:19 pm

Greatest variance: how do you measure this?

Reply



Iyad Ahmed says:

December 20, 2022 at 6:51 am

We compute element wise variance of triangle centers (bounding box centers work as well), then chose the split axis based on the largest component (similar to choosing split axis based on largest component of node's bounding box dimensions),

The naive variance algorithm from here works as a start:

https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance

The 1111Engine.....stl file from here exhibits the issue because it has some long thin triangles:

https://drive.google.com/file/d/1TfpeZHQ9_krY9cSQulYGZ5YbfKdFSL1f/view

Reply



Iyad Ahmed says:

December 20, 2022 at 6:56 am

Here's the essence of it:

We compute element wise variance of triangle centers (bounding box centers work as well), then chose the split axis based on the largest component (similar to choosing split axis based on largest component of node's bounding box dimensions),

Reply



jbikker says:

December 20, 2022 at 8:19 am

Clear! I wasn't sure if you meant actual variance. This sounds like a good alternative to full SAH, thanks for sharing.

Reply



Kudos 🙏



Iyad Ahmed says:

January 24, 2023 at 2:57 pm

Btw here's my C++ code for subdivision based on variance:

<https://github.com/iyadahmed/bvh/blob/Of20de9e3d310816074e79c6bf7c53dc0f78bccc/subdivision.hpp#L13>

It also uses std::partition

also here's the ray/AABB intersection code (currently the bottleneck):

https://github.com/iyadahmed/bvh/blob/Of20de9e3d310816074e79c6bf7c53dc0f78bccc/ray_intersection.hpp#L68

I would be very honored if you checkout the code, thanks again



Iyad Ahmed says:

January 24, 2023 at 2:59 pm

There's one more comment pending moderation



Meow says:

December 30, 2022 at 8:33 pm

```
if (tri[triIdx[i]].centroid[axis] < splitPos) i++;
```

^is this split correct? Isn't it possible for a triangle to have a centroid that's on the left side of the bounding box, but 1 or 2 vertices that put it inside both bounding boxes?

Shouldn't the split be based on whether all three vertices lie on the left side?

Reply



Iyad Ahmed says:

January 24, 2023 at 2:58 pm

it works because we calculate the bounding box based on triangle vertices anyways (not triangle centers)



Chris says:

February 1, 2023 at 6:42 pm

What is the rationale of having a triangle struct store the vertex/centroid data it requires? This would obviously lead to a larger memory footprint as adjacent triangles will be storing duplicate vertex information. Would it be sensible instead to use a vertex buffer and index buffer, where we only need to process each triangle's index and then can fetch the vertex data from a single vertex buffer.

So instead of needing to do: `tri[triIdx].vertex0`

We could do: `vertex_buffer[face_buffer[triIdx][0]]` (in this instance, `face_buffer` is a 2d array of the same format as a face definition in an OBJ file. Conceivably you could convert this to a 1d array with linear index, but I wrote it this way for clarity hopefully).

What is the benefit of having the triangle's explicitly hold their vertices? If I missed this somewhere I apologize for missing it.

Reply



jbikker says:

February 1, 2023 at 9:47 pm

A vertex / index buffer would work fine. It does however add some bandwidth for 'pointer chasing', which is why sometimes the vertices are stored directly. It costs some memory, but it is faster.

Reply



Chris says:

February 6, 2023 at 7:23 pm

Is it always faster? If you were rendering in a coherent way such that adjacent triangles are (often) rendered one after the other, shouldn't previously loaded vertex information already be in cache and thus not need to be read from memory? Or does the fact that each ray traverses the bvh in-between triangle intersections mean that the cache is disrupted with aabb data, and thus vertex information is expelled and no benefit is gained (as I believe there is a benefit in rasterization where each triangle is processed sequentially, and thus cached vertices can help you).

I'm fairly unfamiliar with the gritty details of this sort of thing so I'm hoping my question is sensible!

Reply

jbikker says:

more work than directly accessing (hopefully cached) vertex positions. On top of that, without indirection data there is more room for actual vertex data, so the probability of a cache hit will go up.

This is just in the optimal case that you described. In practice, ray distributions are often not coherent at all, and/or other data access evicts vertex data. In that case every memory access is a cache miss.

To give you a ballpark figure: in practice indexed triangles will make the ray query about 5% slower. This is a rough estimate for packets of primary and shadow rays in the Arauna CPU ray tracer. I have no data for path tracing, but I expect it is similar. It is always faster. Even getting the data from cache is more work than directly accessing (hopefully cached) vertex positions. On top of that, without indirection data there is more room for actual vertex data, so the probability of a cache hit will go up.

This is just in the optimal case that you described. In practice, ray distributions are often not coherent at all, and/or other data access evicts vertex data. In that case every memory access is a cache miss.

To give you a ballpark figure: in practice indexed triangles will make the ray query about 5% slower. This is a rough estimate for packets of primary and shadow rays in the Arauna CPU ray tracer. I have no data for path tracing, but I expect it is similar.

Reply



Panjaksli says:

February 15, 2023 at 6:37 pm

May I ask about the BVH storage in memory ? I cannot read it out from the code... Are two child nodes stored next to each other in the array, or is that a linear storage (meaning first child node is always stored after the parent and the second child is stored after the whole first leaf) ?

I'm using a recursive function which works only at one node at time, resulting in linear storage of tree in array. Should I rewrite it like yours, or do you think it's fine ?

```
void obj_list::split_bvh(uint begin, uint end, uint node_size) {
    aabb bbox = box_from(begin, end);
    uint size = end - begin;
    if (size > node_size) {
        uint n = bvh.size();
        uchar axis = bbox.get_longest_axis();
```

```
bvh.emplace_back(bvh_node(bbox, begin, end, 0));
return;
}
bvh.emplace_back(bvh_node(bbox, n + 1, 0, 1));
split_bvh(begin, split, node_size);
bvh[n].n2 = bvh.size();
split_bvh(split, end, node_size);
}
else bvh.emplace_back(bvh_node(bbox, begin, end, 0));
}
```

Reply



Nico says:

March 8, 2023 at 4:43 pm

Great article! I like all the little optimisation tricks. One note though: By using the centroid you will sometimes assign a primitive to the incorrect side, the bounding box center should be used instead.

Example:

Triangle: a = (-5, 6, 0), b = (-5, 0, 0), c = (7, 0, 0), centroid = (-1, 2, 0), aabbCenter = (1, 3, 0)

Splitting plane: x = 0

Using the centroid, the triangle goes to the left side, which leads to a bleeding of 7 units.

Using the bounding box center, the triangle goes to the right side, which leads to a bleeding of 5 units.

However, centroid can sort some axis-aligned quad cases naturally.

Reply



jBikker says:

March 9, 2023 at 1:04 pm

Thanks for pointing that out, I never realized that it matters which point you pick. But it indeed seems logical that the smallest 'bleeding' as you call it happens with the AABB centroid. TIL. 😊

Reply



Iyad Ahmed says:

May 20, 2023 at 9:17 pm

Great article I have come again to share findings ^^, the partitioning does not work with unsigned integers because j will get decremented below zero and it will wrap around and cause issues ^^, sharing

```
uint32_t i = node->start;
uint32_t j = node->start + node->count - 1;
do
{
while (i <= j &&
triangles[indices[i]].aabb.max.as_array[split_axis] <
split_pos)
{
i++;
}
while (i = split_pos)
{
j--;
}

if (i < j)
{
uint32_t tmp = indices[i];
indices[i] = indices[j];
indices[j] = tmp;
i++;
j--;
}
} while (i < j);
```

Reply



JB says:

May 22, 2023 at 8:06 am

The code does not use unsigned ints in any of the versions in the articles or on GitHub, unless I missed something?

Reply



Iyad Ahmed says:

June 9, 2023 at 6:24 pm

ah sorry I meant people who need need to implement using unsigned ints based on the article, I did and ran into this issue, so posted fix ^^

Reply



As a word of advice to anyone else following this, allowing the BVH to exit construction with a leaf with more tris than the specified threshold is VITAL.

I came upon this article looking for speedups of my implementation of the BVH, and spent 8 hours trying to figure out why my raycasting was 10x slower than the listed speed in pt 2. The singular issue causing this was me forcing my BVH to split until only 4 tris or less remained in each leaf. For such a small detail to cause such a tremendous mess is frankly incredible. It was literally the last thing I checked.

Kudos to you, you saved my bacon on this one. I considered my code trying to select splits that put all tris on only one side of the split a bug. It never would have occurred to me to allow it were not for this article.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐

Save my name, email, and website in this browser for the next time I comment.

POST COMMENT

Home	Projects	Publications	Personal					